🌐 Module 13: IoT Penetration Testing

```
# Exercise 02: Developing a Lua Wireshark Dissector for DICOM Protocol
```

Scenario

Medical IoT Protocol Dissection focuses on analyzing proprietary or medical device network protocols like **DICOM** (Digital Imaging and Communications in Medicine). Developing custom **Lua dissectors** for Wireshark enables penetration testers to parse unencrypted medical image transfers, extract PDU structures, and inspect raw traffic on healthcare IoT networks.

Key technical concepts in DICOM protocol dissection:
1. **Wireshark Lua Subsystem**: Verifying Lua support under **Help -> About Wireshark** and checking protocol dissector tables under **Analyze -> Enabled Protocols**.
2. **DICOM PDU Structure**: Digital Imaging PDU headers contain:
- **PDU Type** (`uint8`): 1=ASSOC Request, 2=ASSOC Accept, 3=ASSOC Reject, 4=Data, 5=RELEASE Request, 6=RELEASE Response, 7=ABORT.
- **Message Length** (`uint16` / `uint32`): Length of payload segment.
3. **Orthanc Medical Server Deployment**: Installing open-source DICOM server Orthanc on Ubuntu (**sudo apt install orthanc**), verifying active DICOM port `4242` and Web API port **0.0.0.0:8042** via `netstat -atn`.
4. **Lua Dissector Scripting**: Constructing custom Wireshark protocol objects using `Proto()`, registering `ProtoField` elements, mapping endianness parsing (`add_le` vs `add`), and attaching to port `4242` via `DissectorTable.get("tcp.port"):add(4242, dicom_protocol)`.

In this exercise, we log into **Ubuntu** (**Ubuntu** / **toor**), open Wireshark, inspect sample capture file **DICOM_C-ECHO-echoscu.pcap**, filter PUSH TCP packets (`tcp.flags.push == 1`), follow TCP stream, install **Orthanc** server, verify listening interface **0.0.0.0:8042**, write a custom Lua dissector script, and compare packet rendering across Ubuntu and Windows Server 2022.

Objective

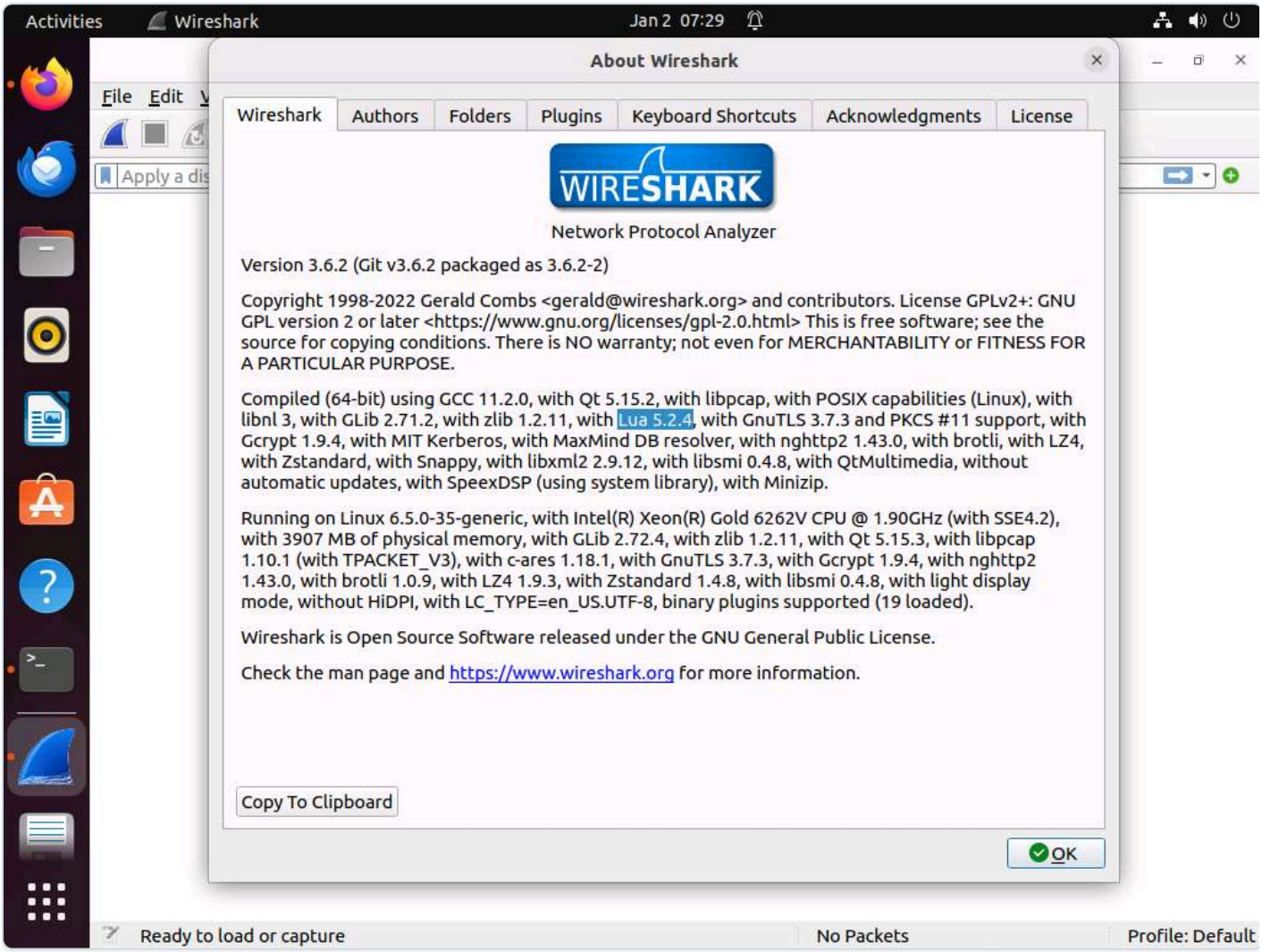
The objective of this lab is to help students learn how to:
- Authenticate to Ubuntu target VM (**Ubuntu** / **toor**)
- Verify Wireshark Lua plugin support (**Help -> About Wireshark**)
- Verify DICOM dissector activation (**Analyze -> Enabled Protocols**)
- Inspect PCAP packet capture file **DICOM_C-ECHO-echoscu.pcap**
- Apply TCP push flag display filter: **tcp.flags.push == 1**
- Follow TCP Stream to inspect raw unencrypted DICOM A-Type messages
- Install Orthanc DICOM server: **sudo apt install orthanc**
- Verify Orthanc Web API listening address: **netstat -atn** (**0.0.0.0:8042** & port `4242`)
- Access Orthanc web interface using credentials (**orthanc** / **orthanc**)
- Develop and register custom Lua Wireshark dissector script for DICOM TCP port `4242`
- Compare packet dissection rendering against Windows Server 2022 (**Administrator** / **Pa$$w0rd**)

Task 1: Verify Wireshark Lua Support, Analyze DICOM PCAP, Deploy Orthanc, and Write Lua Dissector

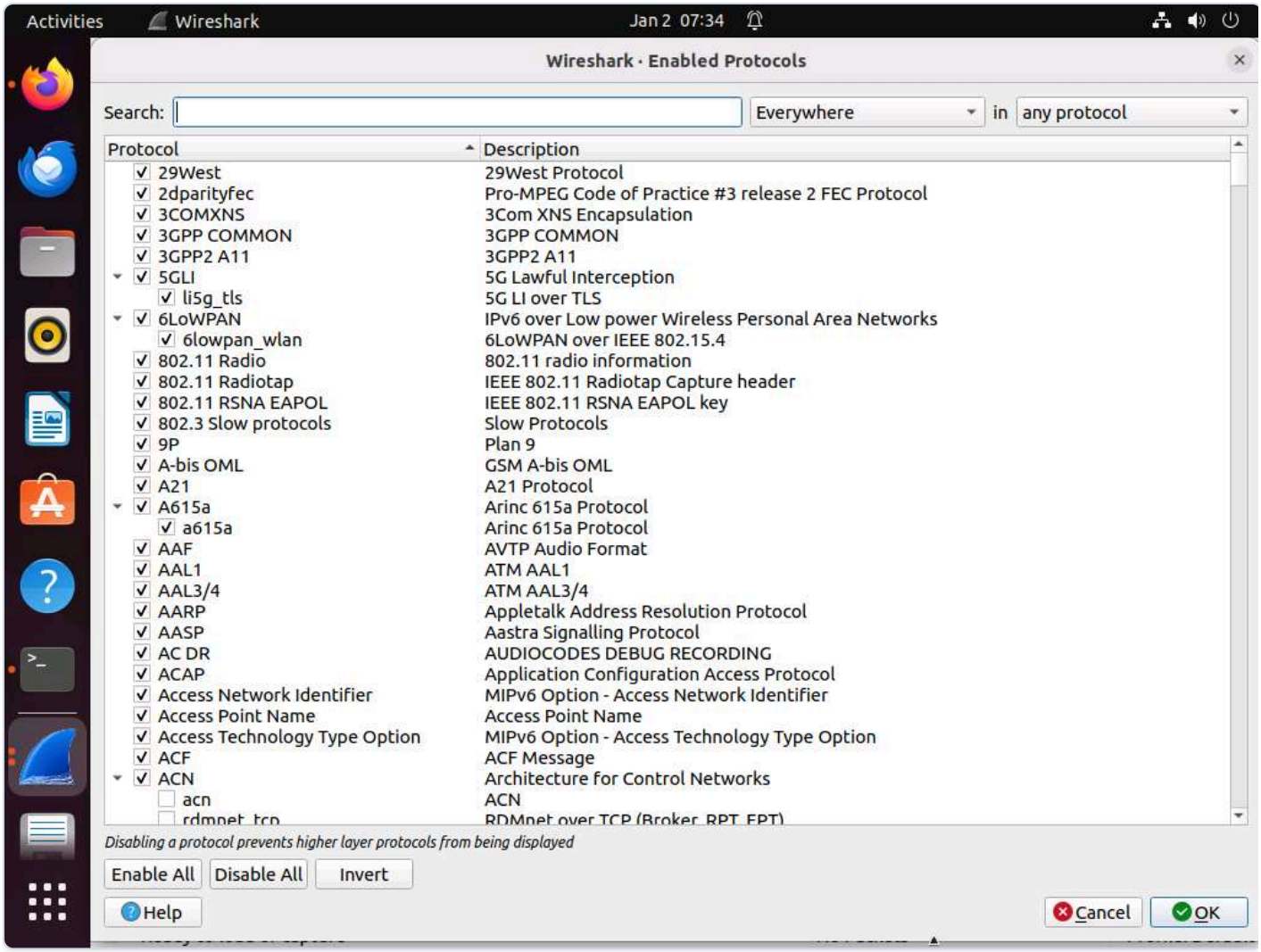
In this task, we inspect Wireshark Lua settings, filter `DICOM_C-ECHO-echoscu.pcap`, install Orthanc server, verify Web API port `0.0.0.0:8042`, write custom DICOM Lua dissector code, and compare dissector output across operating systems.

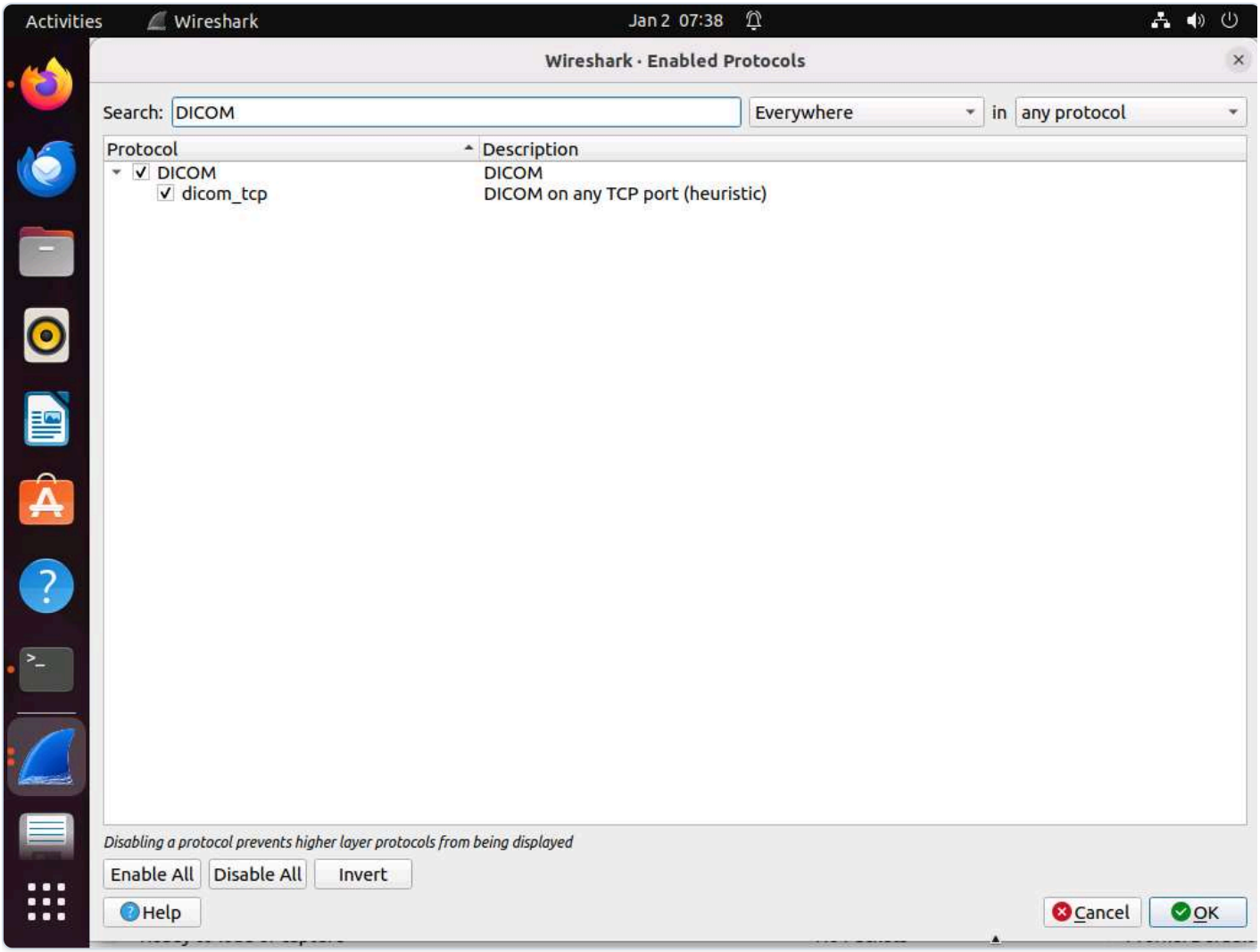
Lab Tasks

1. Log into **Ubuntu** VM using credentials (**Ubuntu** / **toor**). Open Wireshark with `sudo` and verify Lua is enabled under **Help -> About Wireshark**.

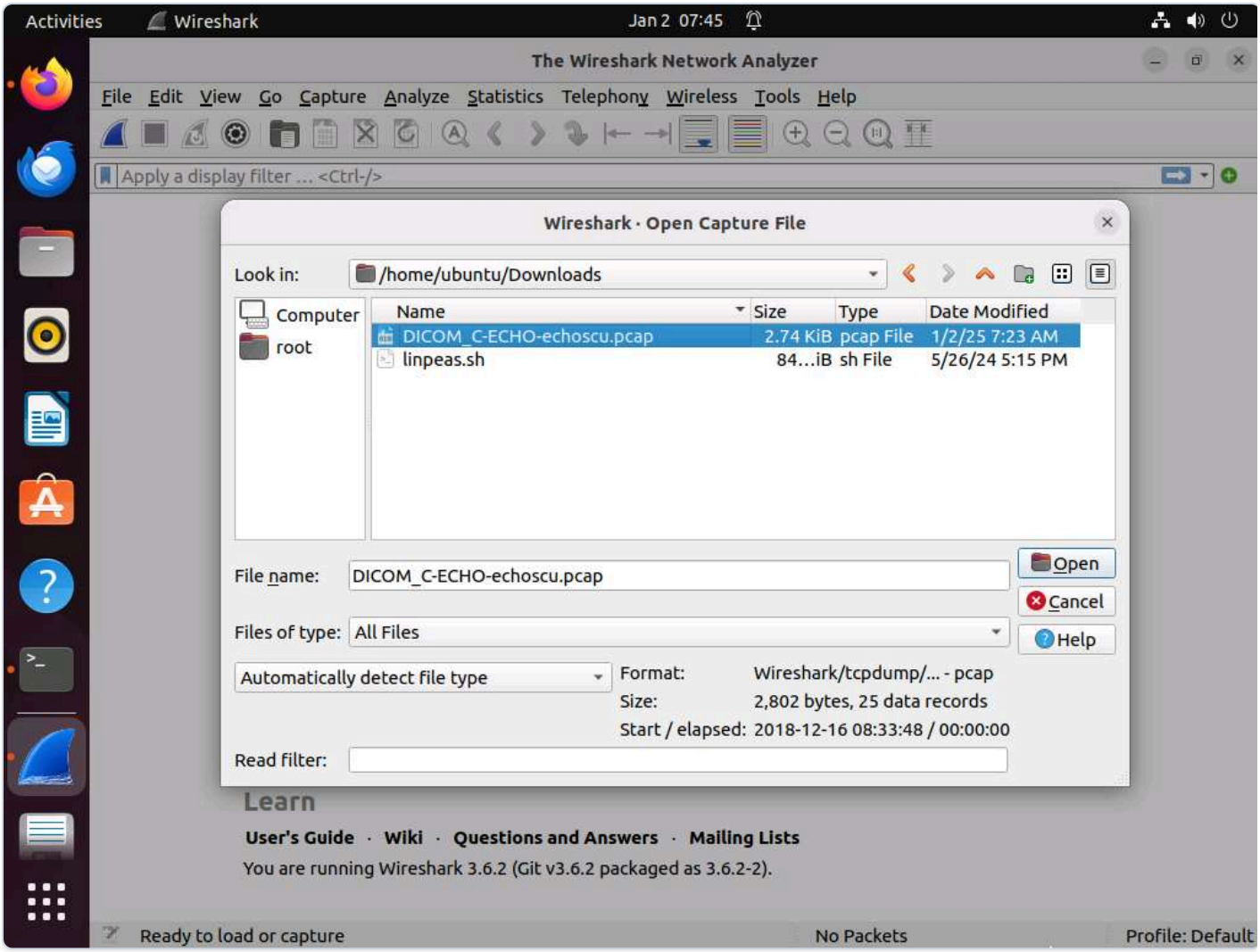


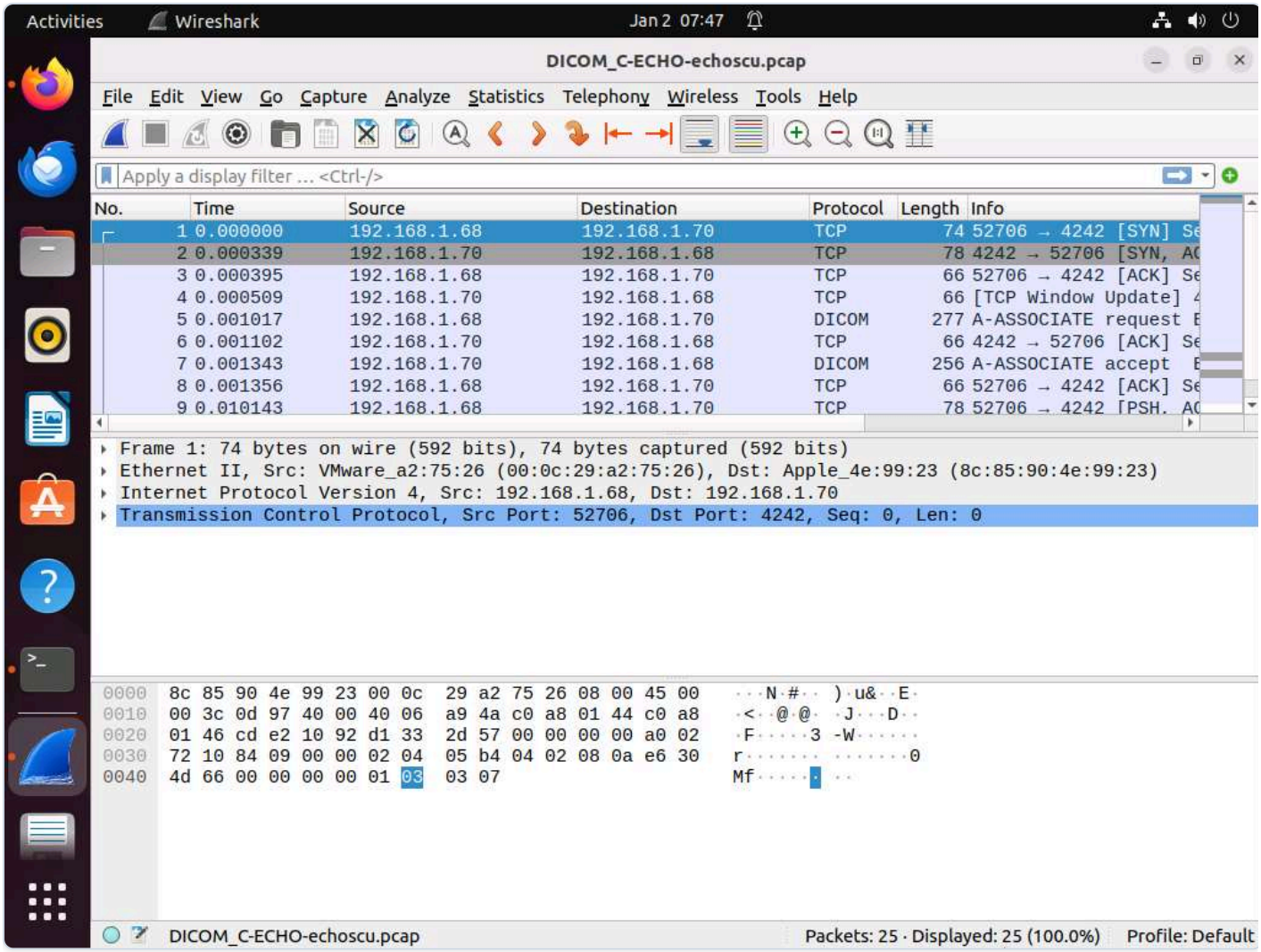
1. Navigate to **Analyze -> Enabled Protocols** and verify DICOM protocol dissector status.


Activities
Wireshark
Jan 2 07:34
Wireshark · Enabled Protocols
Search:
Everywhere
in
any protocol
Protocol
Description
29West
29West Protocol
2dparityfec
Pro-MPEG Code of Practice #3 release 2 FEC Protocol
3COMXNS
3Com XNS Encapsulation
3GPP COMMON
3GPP COMMON
3GPP2 A11
3GPP2 A11
5GLI
5G Lawful Interception
li5g_tls
5G LI over TLS
6LoWPAN
IPv6 over Low power Wireless Personal Area Networks
6lowpan_wlan
6LoWPAN over IEEE 802.15.4
802.11 Radio
802.11 radio information
802.11 Radiotap
IEEE 802.11 Radiotap Capture header
802.11 RSNA EAPOL
IEEE 802.11 RSNA EAPOL key
802.3 Slow protocols
Slow Protocols
9P
Plan 9
A-bis OML
GSM A-bis OML
A21
A21 Protocol
A615a
Arinc 615a Protocol
a615a
Arinc 615a Protocol
AAF
AVTP Audio Format
AAL1
ATM AAL1
AAL3/4
ATM AAL3/4
AARP
Appletalk Address Resolution Protocol
AASP
Aastra Signalling Protocol
AC DR
AUDIOCODES DEBUG RECORDING
ACAP
Application Configuration Access Protocol
Access Network Identifier
MIPv6 Option - Access Network Identifier
Access Point Name
Access Point Name
Access Technology Type Option
MIPv6 Option - Access Technology Type Option
ACF
ACF Message
ACN
Architecture for Control Networks
acn
ACN
Disabling a protocol prevents higher layer protocols from being displayed
Enable All
Disable All
Invert
Help
Cancel
OK

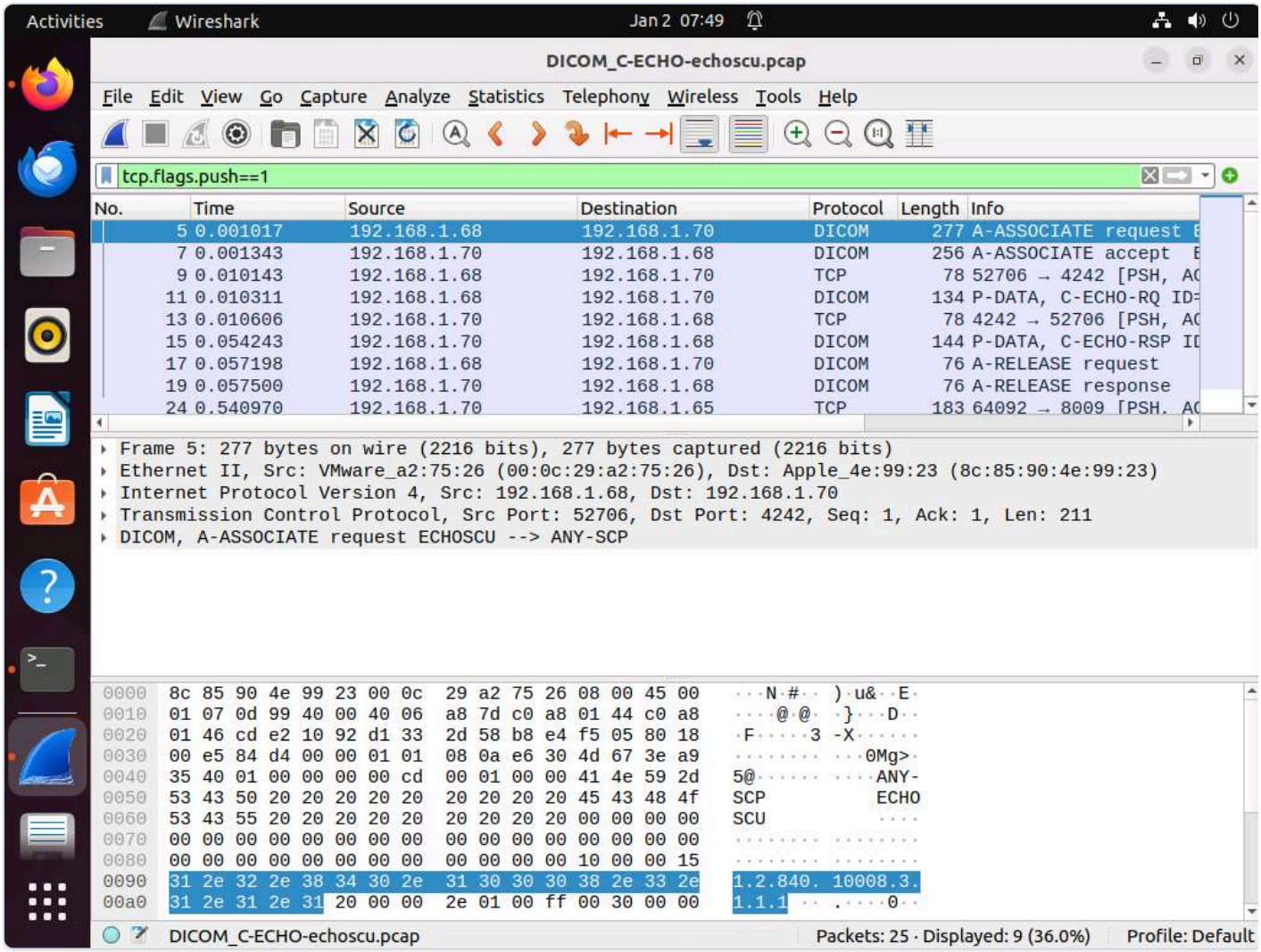


1. Open sample capture file **DICOM_C-ECHO-echoscu.pcap** from `Downloads` directory.

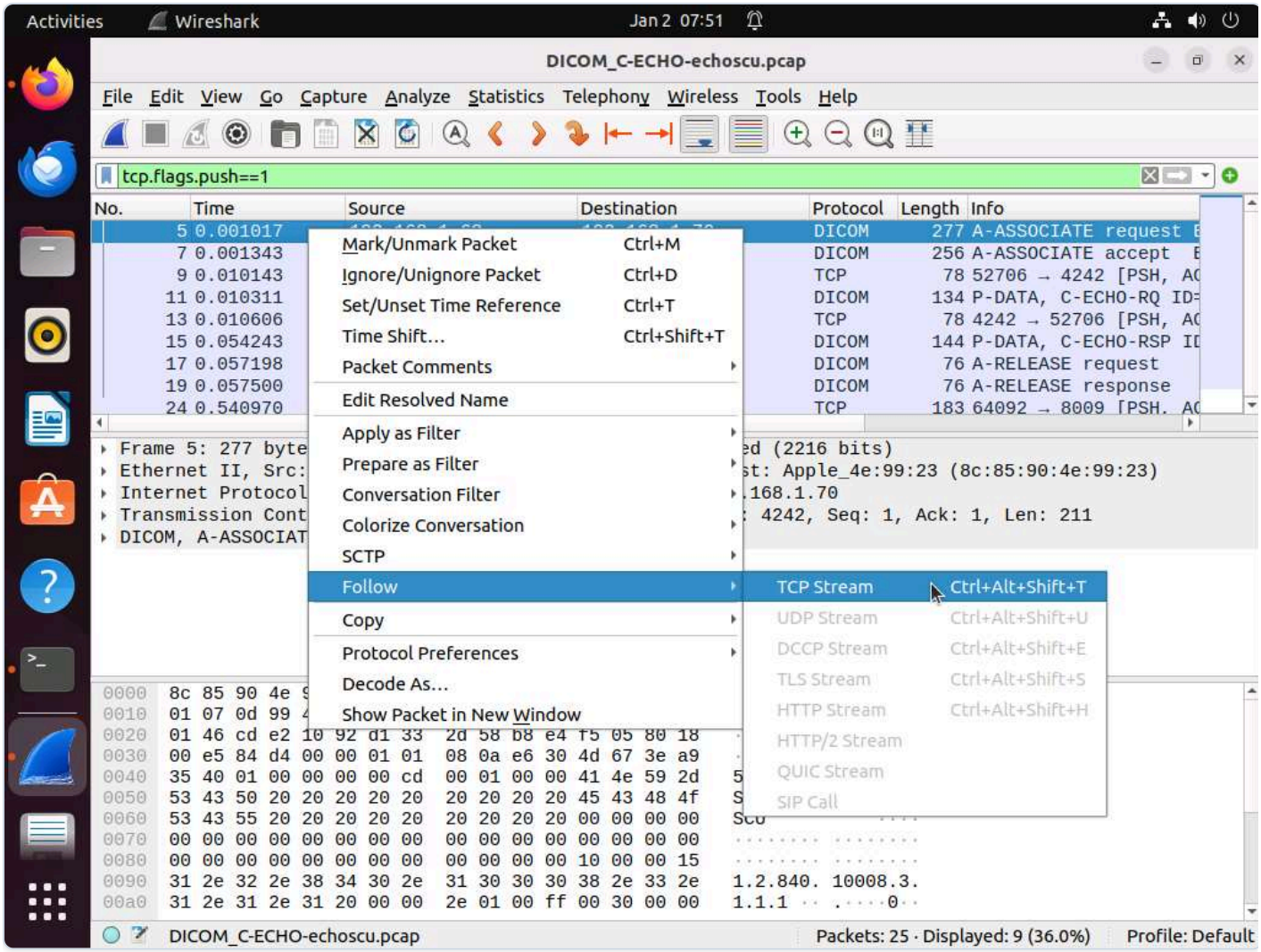

Activities
Wireshark
Jan 2 07:45
The Wireshark Network Analyzer
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
Wireshark · Open Capture File
Look in:
/home/ubuntu/Downloads
Computer
root
Name
Size
Type
Date Modified
DICOM_C-ECHO-echoscu.pcap
2.74 KiB
pcap File
1/2/25 7:23 AM
linpeas.sh
84...iB
sh File
5/26/24 5:15 PM
File name:
DICOM_C-ECHO-echoscu.pcap
Open
Cancel
Help
Files of type:
All Files
Automatically detect file type
Format:
Wireshark/tcpdump/... - pcap
Size:
2,802 bytes, 25 data records
Start / elapsed:
2018-12-16 08:33:48 / 00:00:00
Read filter:
Learn
User's Guide · Wiki · Questions and Answers · Mailing Lists
You are running Wireshark 3.6.2 (Git v3.6.2 packaged as 3.6.2-2).
Ready to load or capture
No Packets
Profile: Default

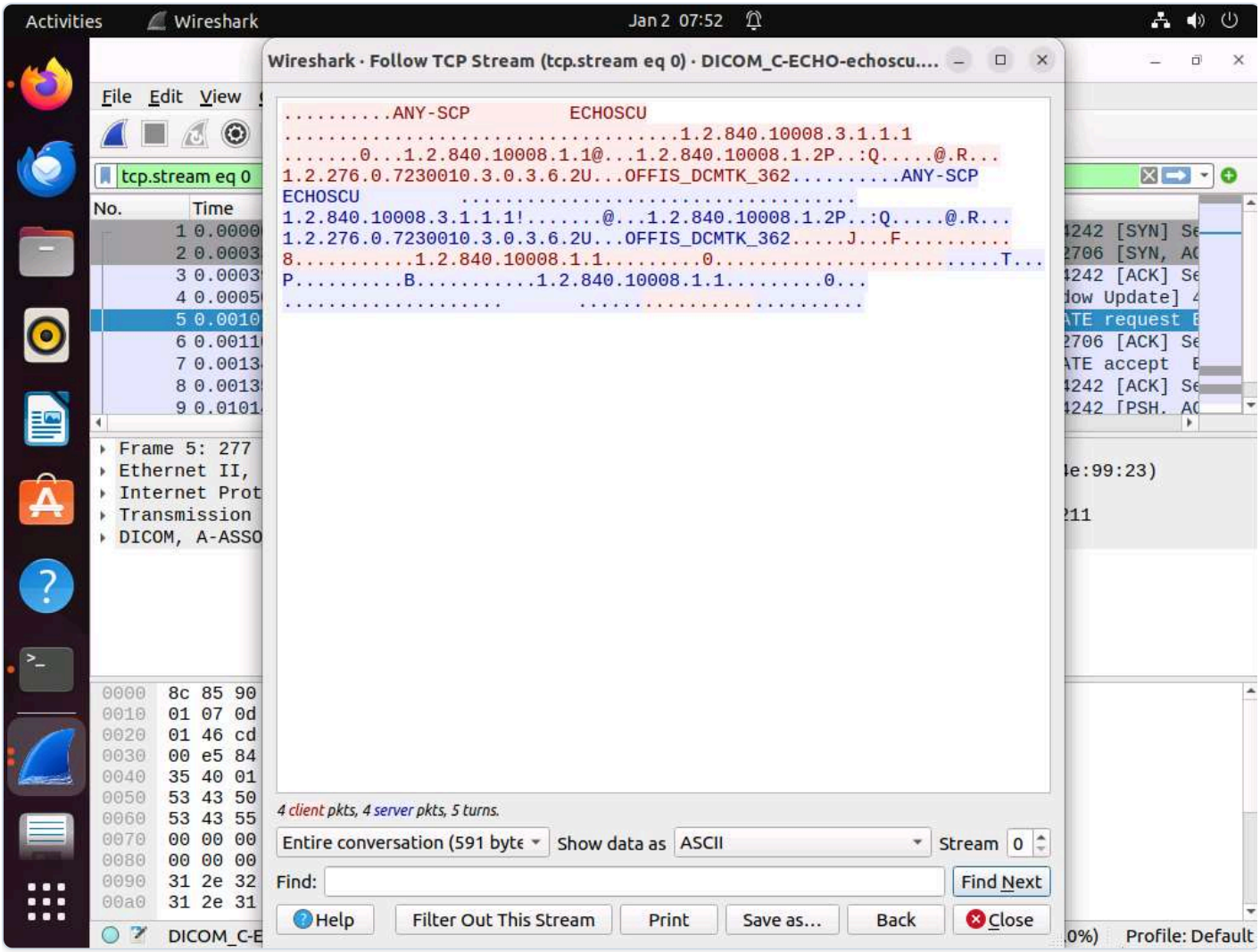


1. Apply display filter to isolate TCP PUSH packets:
   tcp.flags.push == 1



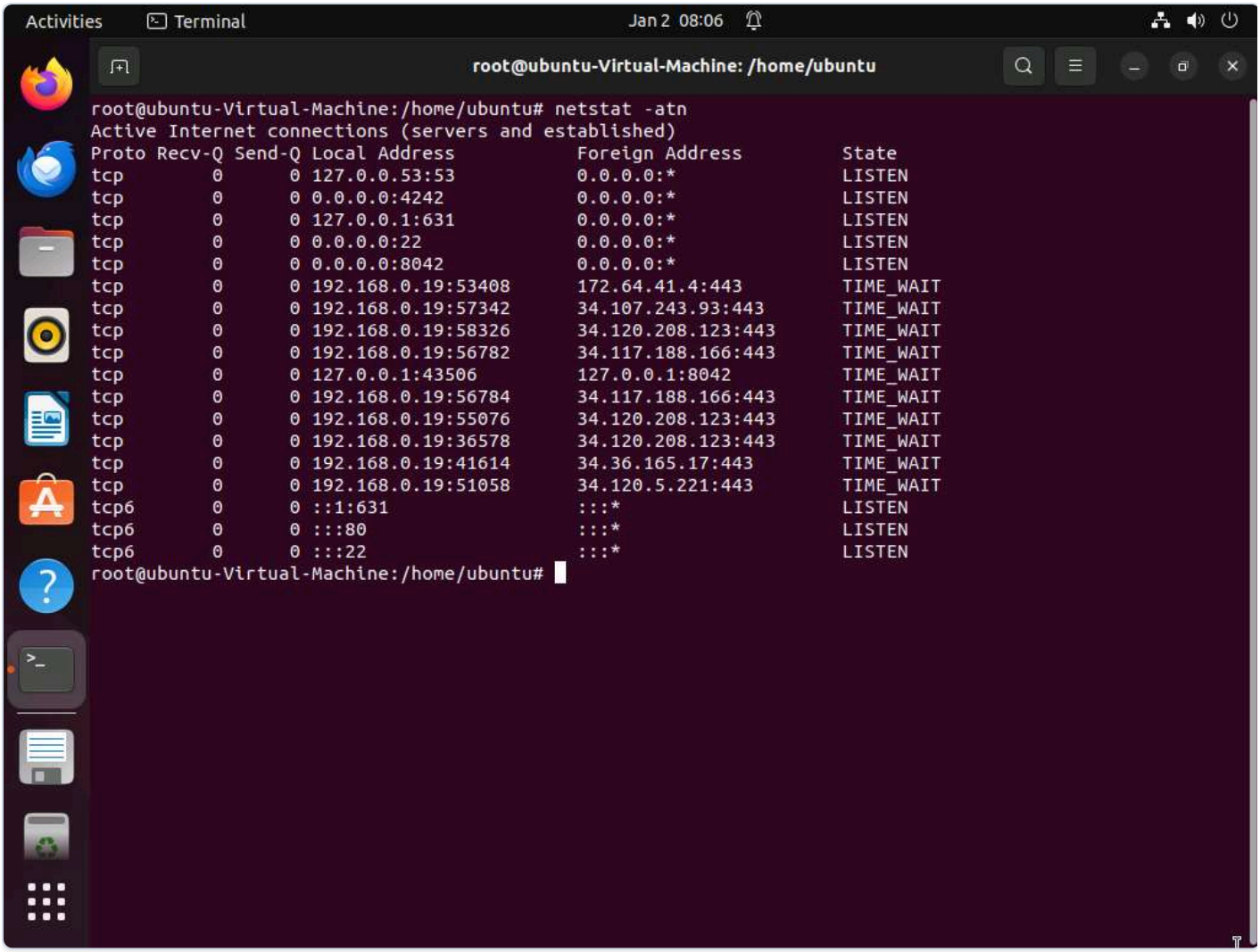
1. Right-click packet and select **Follow -> TCP Stream** to inspect raw payload data.


Activities
Wireshark
Jan 2 07:51
DICOM_C-ECHO-echoscu.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.flags.push==1
No. Time Source Destination Protocol Length Info
5 0.001017 DICOM 277 A-ASSOCIATE request
7 0.001343 DICOM 256 A-ASSOCIATE accept
9 0.010143 TCP 78 52706 → 4242 [PSH, AC
11 0.010311 DICOM 134 P-DATA, C-ECHO-RQ ID=
13 0.010606 TCP 78 4242 → 52706 [PSH, AC
15 0.054243 DICOM 144 P-DATA, C-ECHO-RSP ID
17 0.057198 DICOM 76 A-RELEASE request
19 0.057500 DICOM 76 A-RELEASE response
24 0.540970 TCP 183 64092 → 8009 [PSH, AC
Frame 5: 277 byte
Ethernet II, Src:
Internet Protocol
Transmission Cont
DICOM, A-ASSOCIAT
ed (2216 bits)
st: Apple_4e:99:23 (8c:85:90:4e:99:23)
.168.1.70
: 4242, Seq: 1, Ack: 1, Len: 211
Mark/Unmark Packet Ctrl+M
Ignore/Unignore Packet Ctrl+D
Set/Unset Time Reference Ctrl+T
Time Shift... Ctrl+Shift+T
Packet Comments
Edit Resolved Name
Apply as Filter
Prepare as Filter
Conversation Filter
Colorize Conversation
SCTP
Follow
Copy
Protocol Preferences
Decode As...
Show Packet in New Window
TCP Stream Ctrl+Alt+Shift+T
UDP Stream Ctrl+Alt+Shift+U
DCCP Stream Ctrl+Alt+Shift+E
TLS Stream Ctrl+Alt+Shift+S
HTTP Stream Ctrl+Alt+Shift+H
HTTP/2 Stream
QUIC Stream
SIP Call
0000 8c 85 90 4e
0010 01 07 0d 99
0020 01 46 cd e2 10 92 d1 33 2d 58 b8 e4 f5 05 80 18
0030 00 e5 84 d4 00 00 01 01 08 0a e6 30 4d 67 3e a9
0040 35 40 01 00 00 00 00 cd 00 01 00 00 41 4e 59 2d
0050 53 43 50 20 20 20 20 20 20 20 20 20 45 43 48 4f
0060 53 43 55 20 20 20 20 20 20 20 20 20 00 00 00 00
0070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0080 00 00 00 00 00 00 00 00 00 00 00 00 10 00 00 15
0090 31 2e 32 2e 38 34 30 2e 31 30 30 30 38 2e 33 2e 1.2.840. 10008.3.
00a0 31 2e 31 2e 31 20 00 00 2e 01 00 ff 00 30 00 00 1.1.1 ·· .····0··
DICOM_C-ECHO-echoscu.pcap
Packets: 25 · Displayed: 9 (36.0%)
Profile: Default

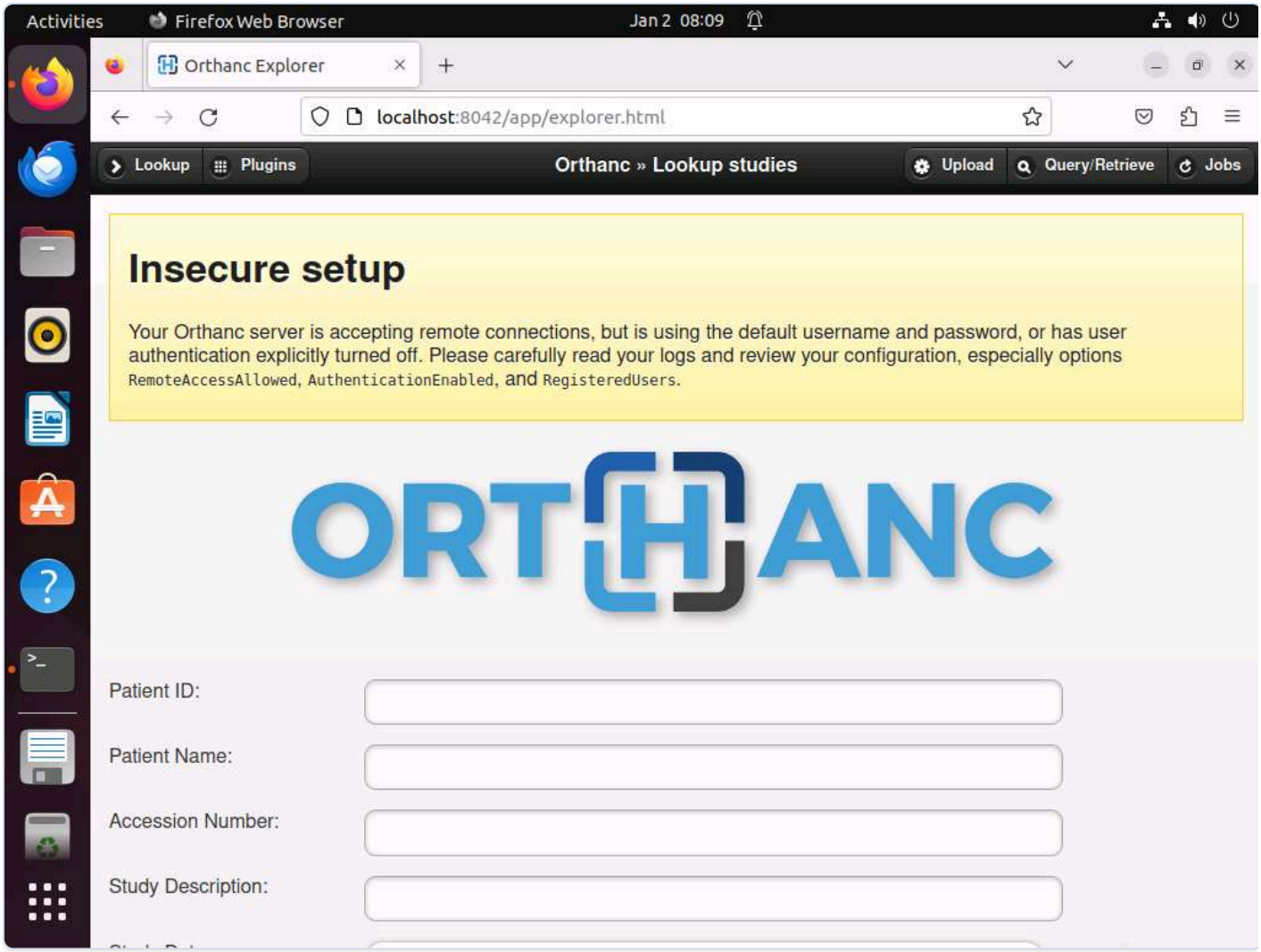


1. Install Orthanc open-source DICOM medical server:
   sudo apt install orthanc

2. Verify listening TCP ports for DICOM (`4242`) and Web API (**0.0.0.0:8042**):
   netstat -atn



1. Open browser to `http://localhost:8042` and authenticate with credentials (**orthanc** / **orthanc**). Start Wireshark capture on `any` interface.



1. Write custom Lua dissector script for DICOM PDU parsing:

```
dicom_protocol = Proto("dicom-a", "DICOM A-Type message")

pdu_type = ProtoField.uint8("dicom-a.pdu_type", "pduType", base.DEC, {
    [1] = "ASSOC Request",
    [2] = "ASSOC Accept",
    [3] = "ASSOC Reject",
    [4] = "Data",
    [5] = "RELEASE Request",
    [6] = "RELEASE Response",
    [7] = "ABORT"
})

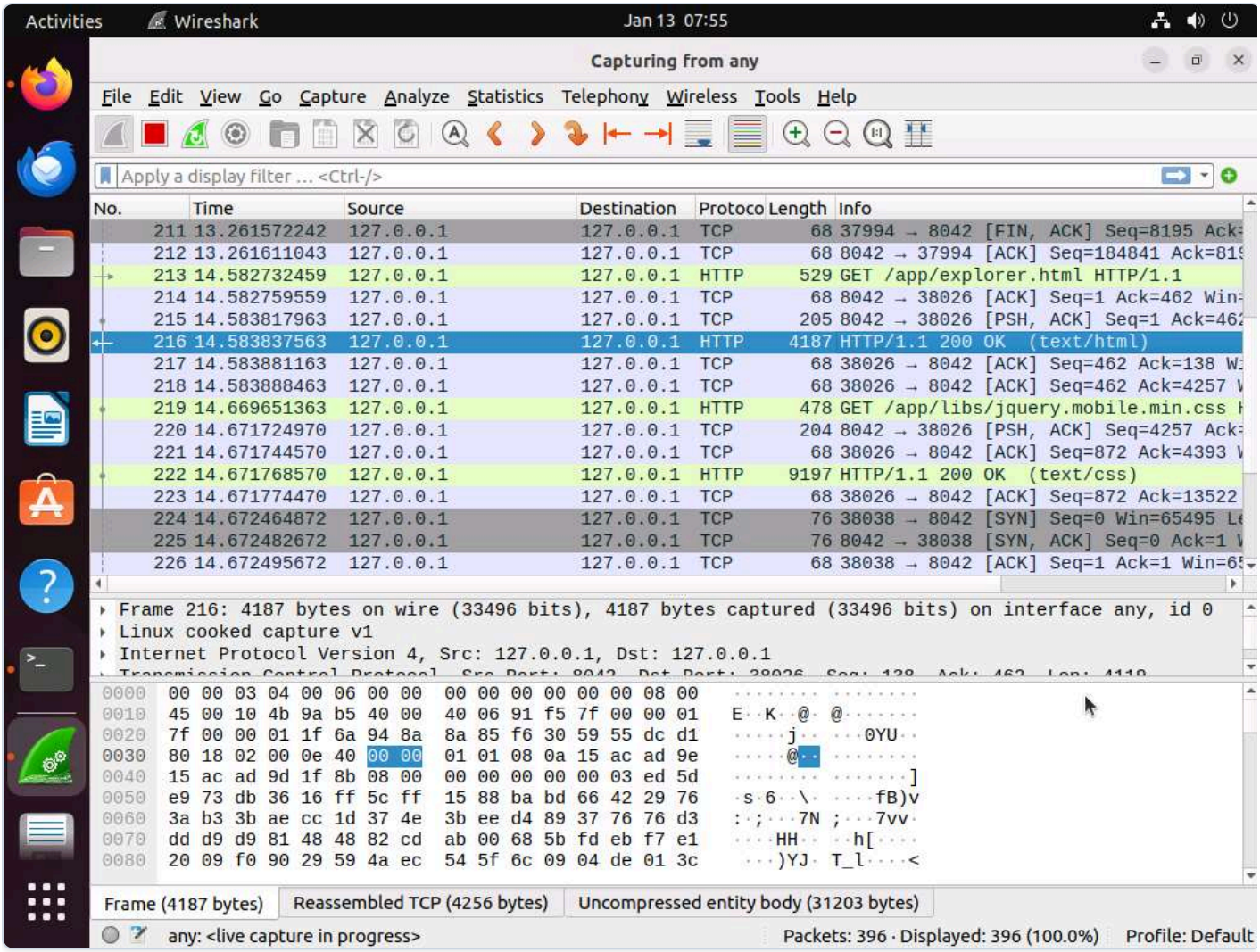
message_length = ProtoField.uint16("dicom-a.message_length", "messageLength", base.DEC)

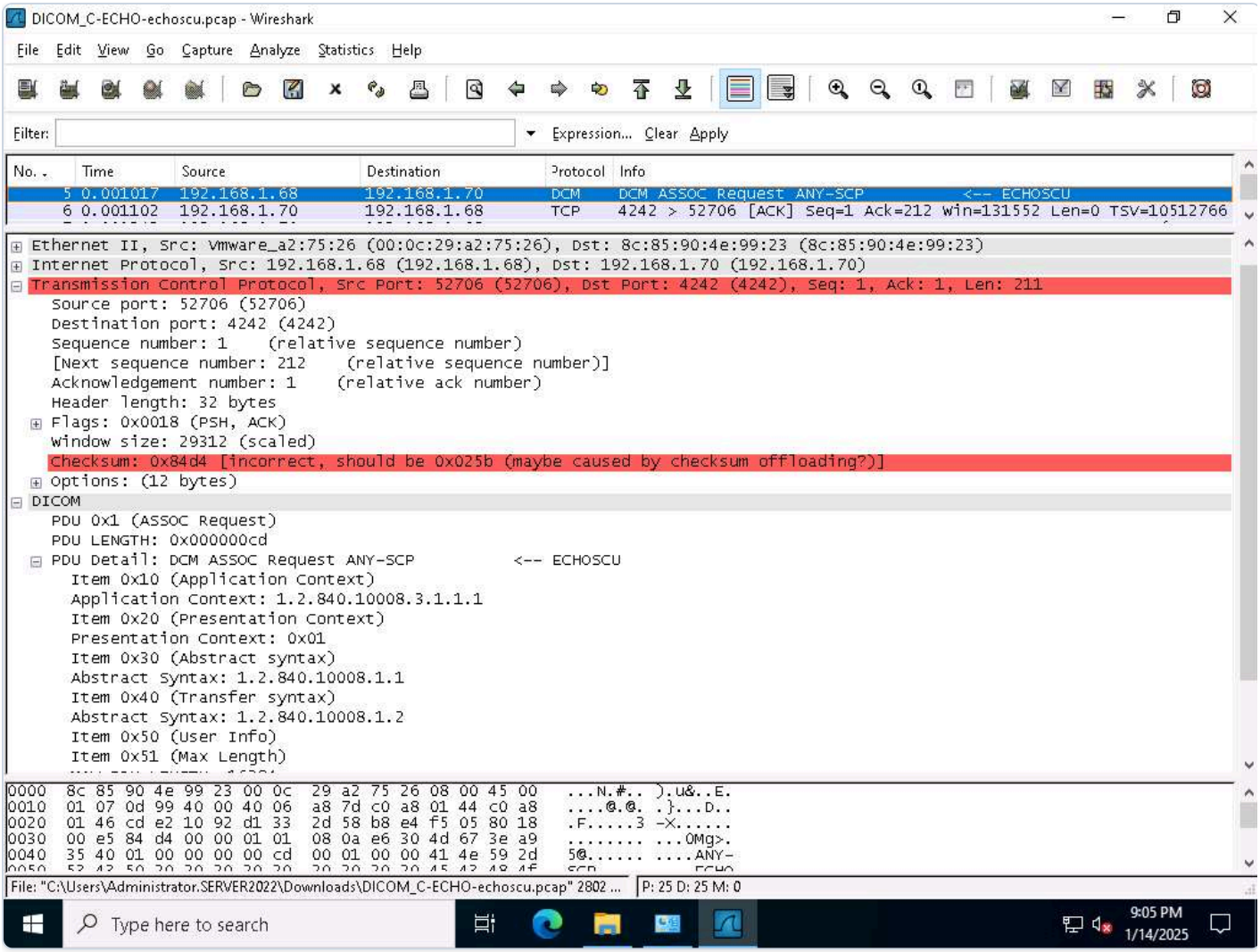
dicom_protocol.fields = {pdu_type, message_length}

function dicom_protocol.dissector(buffer, pinfo, tree)
    pinfo.cols.protocol = dicom_protocol.name
    local subtree = tree:add(dicom_protocol, buffer(), "DICOM PDU")
    subtree:add_le(pdu_type, buffer(0,1))
    subtree:add(message_length, buffer(2,4))
end

local tcp_port = DissectorTable.get("tcp.port")
tcp_port:add(4242, dicom_protocol)
```

1. Switch to **Windows Server 2022** VM (**Administrator** / **Pa$$w0rd**), open Wireshark, load `DICOM_C-ECHO-echoscu.pcap`, and compare default packet dissection against custom Lua dissector on Ubuntu.


Activities
Wireshark
Jan 13 07:55
Capturing from any
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protoco Length Info
211 13.261572242 127.0.0.1 127.0.0.1 TCP 68 37994 → 8042 [FIN, ACK] Seq=8195 Ack=
212 13.261611043 127.0.0.1 127.0.0.1 TCP 68 8042 → 37994 [ACK] Seq=184841 Ack=819
213 14.582732459 127.0.0.1 127.0.0.1 HTTP 529 GET /app/explorer.html HTTP/1.1
214 14.582759559 127.0.0.1 127.0.0.1 TCP 68 8042 → 38026 [ACK] Seq=1 Ack=462 Win=
215 14.583817963 127.0.0.1 127.0.0.1 TCP 205 8042 → 38026 [PSH, ACK] Seq=1 Ack=462
216 14.583837563 127.0.0.1 127.0.0.1 HTTP 4187 HTTP/1.1 200 OK (text/html)
217 14.583881163 127.0.0.1 127.0.0.1 TCP 68 38026 → 8042 [ACK] Seq=462 Ack=138 W
218 14.583888463 127.0.0.1 127.0.0.1 TCP 68 38026 → 8042 [ACK] Seq=462 Ack=4257 W
219 14.669651363 127.0.0.1 127.0.0.1 HTTP 478 GET /app/libs/jquery.mobile.min.css H
220 14.671724970 127.0.0.1 127.0.0.1 TCP 204 8042 → 38026 [PSH, ACK] Seq=4257 Ack=
221 14.671744570 127.0.0.1 127.0.0.1 TCP 68 38026 → 8042 [ACK] Seq=872 Ack=4393 W
222 14.671768570 127.0.0.1 127.0.0.1 HTTP 9197 HTTP/1.1 200 OK (text/css)
223 14.671774470 127.0.0.1 127.0.0.1 TCP 68 38026 → 8042 [ACK] Seq=872 Ack=13522
224 14.672464872 127.0.0.1 127.0.0.1 TCP 76 38038 → 8042 [SYN] Seq=0 Win=65495 Le
225 14.672482672 127.0.0.1 127.0.0.1 TCP 76 8042 → 38038 [SYN, ACK] Seq=0 Ack=1 W
226 14.672495672 127.0.0.1 127.0.0.1 TCP 68 38038 → 8042 [ACK] Seq=1 Ack=1 Win=65
Frame 216: 4187 bytes on wire (33496 bits), 4187 bytes captured (33496 bits) on interface any, id 0
Linux cooked capture v1
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
0000 00 00 03 04 00 06 00 00 00 00 00 00 00 00 08 00
0010 45 00 10 4b 9a b5 40 00 40 06 91 f5 7f 00 00 01
0020 7f 00 00 01 1f 6a 94 8a 8a 85 f6 30 59 55 dc d1
0030 80 18 02 00 0e 40 00 00 01 01 08 0a 15 ac ad 9e
0040 15 ac ad 9d 1f 8b 08 00 00 00 00 00 00 03 ed 5d
0050 e9 73 db 36 16 ff 5c ff 15 88 ba bd 66 42 29 76
0060 3a b3 3b ae cc 1d 37 4e 3b ee d4 89 37 76 76 d3
0070 dd d9 d9 81 48 48 82 cd ab 00 68 5b fd eb f7 e1
0080 20 09 f0 90 29 59 4a ec 54 5f 6c 09 04 de 01 3c
Frame (4187 bytes)
Reassembled TCP (4256 bytes)
Uncompressed entity body (31203 bytes)
any: <live capture in progress>
Packets: 396 · Displayed: 396 (100.0%)
Profile: Default



This concludes the lab exercise.

IMPORTANT: ❓ **Question 13.2.1.1** Utilize Parrot Security, Ubuntu, and Windows Server 2022 machines to develop a Lua-based Wireshark dissector for the DICOM protocol and analyze DICOM traffic. Install Orthanc on the Ubuntu machine, identify the HTTP API and Web Interface port utilized by Orthanc. [!SUCCESS] ✅ **Answer:** **0.0.0.0:8042**